Before Launch Checklist

How to use this playbook

Use this after development is complete, as a Production Readiness Review one week before go-live. Nothing goes live until it is signed off by the tech lead. If an item does not apply, write "N/A" and the reason next to it instead of skipping it. Planning and setup items live in the separate Before Development Checklist.

The checklists combine how the biggest engineering companies ship software:

Company	Practice	What we borrow
Google	SRE Production Readiness Review (PRR), error budgets, blameless postmortems	Formal go-live review, SLOs, incident learning
Amazon / AWS	Well-Architected Framework (6 pillars), "working backwards" docs, two-way vs one-way door decisions	Design review, cost and reliability checks
Microsoft	Security Development Lifecycle (SDL), threat modeling	Security before code, not after
Netflix	Chaos engineering, canary releases	Test failure on purpose, release to a few users first
Meta / Uber	Feature flags, gradual rollouts, kill switches	Turn features off without redeploying
OWASP	ASVS and Top 10	The security checklist items
DORA research	Deployment frequency, lead time, change failure rate, restore time	How we measure the team

The golden rule from all of them: **plan for failure, automate the boring checks, and make every change reversible.**

Production Readiness Review

Run this as a meeting with the whole team one week before go-live. Any unticked item in B2, B3, B4 or B7 blocks the launch.

B1. Code quality

- [ ] All PRs reviewed; no open TODO, FIXME or commented-out code in critical paths
- [ ] No console.log / print debugging left; proper log levels used
- [ ] Error handling everywhere: no empty catch blocks, user sees a friendly message, logs get the details
- [ ] Timeouts set on every external call (HTTP, database, queue)
- [ ] Pagination on every list API; no "return all rows"
- [ ] Input validation on the server side (never trust only the frontend)

B2. Testing

- [ ] Unit tests for business logic; coverage target met (e.g. 70%+ on core modules)
- [ ] Integration tests for APIs, database and queues
- [ ] End-to-end tests for top user journeys: sign up, login, main action, payment, logout
- [ ] Load test at 2–3x expected peak traffic (k6, JMeter, Locust), results recorded
- [ ] Tested on real devices and browsers the users actually use, including slow 3G
- [ ] Regression test after the final build; UAT sign-off from product or client
- [ ] Database migration tested on a copy of production-size data

B3. Security

- [ ] OWASP Top 10 checked: injection, XSS, broken auth, IDOR, SSRF, misconfiguration
- [ ] VAPT or pen test done; all critical and high findings fixed
- [ ] Dependency scan clean of critical vulnerabilities
- [ ] Secrets rotated before launch; no secrets in git history
- [ ] HTTPS only, HSTS, secure headers (CSP, X-Frame-Options), cookies Secure + HttpOnly + SameSite
- [ ] CORS allows only our domains
- [ ] Rate limiting and brute-force protection on login, OTP and public APIs
- [ ] Admin panels not public; protected by VPN, IP allowlist or SSO + MFA
- [ ] Debug mode, Swagger UI and test endpoints disabled in production
- [ ] Least privilege for service accounts, database users and cloud IAM

B4. Infrastructure and reliability

- [ ] SLOs defined (e.g. 99.9% uptime, p95 latency under 300 ms)
- [ ] Health, readiness and liveness probes configured
- [ ] Autoscaling (HPA) and CPU/memory requests and limits on every pod

- [ ] At least 2 replicas of every critical service across availability zones
- [ ] Automated database backups, and a restore actually tested
- [ ] RTO and RPO agreed and proven by a drill
- [ ] Queues have retries with backoff and a dead-letter queue
- [ ] Graceful shutdown: in-flight requests and messages finish before a pod stops
- [ ] CDN and caching for static files
- [ ] Cloud budget alerts set

B5. Observability

- [ ] Centralized logs with request IDs that follow one request across services
- [ ] Metrics dashboards: traffic, errors, latency, saturation (Google's 4 golden signals)
- [ ] Distributed tracing (OpenTelemetry)
- [ ] Error tracking (Sentry or App Insights) with alerts
- [ ] Alerts go to a named on-call person, not a channel nobody reads
- [ ] Uptime monitor from outside our network
- [ ] Logs retained 180 days (CERT-In) and contain no PII

B6. Data and compliance

- [ ] Privacy Policy, Terms of Service and Cookie Policy live and linked
- [ ] Consent captured and stored with timestamp
- [ ] Delete-account and export-data flows working end to end
- [ ] Retention jobs running
- [ ] Vendor DPAs signed (cloud, SMS, email, payment, analytics)
- [ ] Breach response plan: who reports to CERT-In within 6 hours
- [ ] Grievance officer contact published (if users post content)
- [ ] Accessibility checked (WCAG 2.1 AA) for EU/UK users
- [ ] Open-source licences checked (no GPL code in closed products without approval)

B7. Release plan

- [ ] Release checklist and runbook written step by step
- [ ] Rollback tested: one command or one click, under 10 minutes
- [ ] Canary or blue-green deployment; start with 5–10% of traffic
- [ ] Feature flags and a kill switch for risky features
- [ ] Database migrations backward compatible (expand, then contract)
- [ ] Launch window chosen: not Friday evening, not before a holiday

- [ ] DNS TTL lowered a day before, SSL certificates valid with auto-renewal
- [ ] Go / no-go meeting held and decision recorded

B8. Support and operations

- [ ] On-call rota for launch week, with phone numbers
- [ ] Incident runbook: severity levels, who to call, how to communicate
- [ ] Status page or a way to tell users about outages
- [ ] Support channel ready and support team briefed
- [ ] Analytics and business metrics tracked from minute one
- [ ] User docs, FAQ and release notes published

B9. Frontend, mobile and performance

- [ ] Production build minified, source maps not public
- [ ] Page speed checked (Lighthouse score, Core Web Vitals) on mobile
- [ ] Images compressed and lazy-loaded
- [ ] Loading, empty and error states on every screen
- [ ] Forms prevent double submit
- [ ] SEO basics: titles, meta tags, sitemap, robots.txt (block staging from search engines)
- [ ] Favicon, app icons, 404 and 500 pages
- [ ] Mobile apps: store listing, privacy labels, screenshots, review guidelines checked
- [ ] Crash reporting in apps (Firebase Crashlytics or Sentry)
- [ ] Force-update mechanism for old app versions
- [ ] Push notifications tested on real iOS and Android devices

Silly mistakes that take down production

Most real outages come from small, boring mistakes, not hard problems. Teach these first.

Mistake	What happens	Simple fix
.env or API key pushed to GitHub	Bots find it in minutes; cloud bill or data leak	Secret scanning in CI, .gitignore, rotate immediately if leaked
Running a migration or DELETE on production without WHERE	Whole table wiped	Read-only prod access by default; review every prod query; tested backups

Mistake	What happens	Simple fix
Testing against the production database	Real customers get test emails, SMS or charges	Separate DBs and keys per environment
No timeout on an external API call	One slow vendor freezes the whole app	Timeouts + circuit breaker on every call
Missing database index	App fast with 100 rows, dead with 1 million	Check query plans; load test with real-size data
SSL certificate or domain expired	Site shows "not secure" or goes down	Auto-renew + expiry alerts 30 days ahead
Disk full from logs	Service crashes silently	Log rotation, retention limits, disk alerts
Timezone bugs	Wrong dates, missed bookings, double charges	Store UTC, convert only for display
Same webhook or message processed twice	Customer charged twice	Idempotency keys
Deploy on Friday evening	Bug found when nobody is around	Deploy early in the week, during work hours
No rollback plan	Small bug becomes hours of downtime	Practice rollback before launch
Unlimited file upload size	Storage fills, server crashes	Size and type limits
Error messages show stack traces	Attackers learn your internals	Generic message to user, details in logs
CORS set to * or debug mode on	Open door for attacks	Production config review in B3
Backups never restored	Backup is empty or corrupt when needed	Monthly restore drill
Hard-coded URLs, IPs or limits	Breaks when moving environments	Use config and environment variables

After launch: Day 1 and ongoing habits

Launch is the start, not the finish. The first 48 hours decide whether users trust the product.

First 48 hours

- [ ] Team watches dashboards and error tracker in shifts
- [ ] Compare live error rate and latency against the SLOs every few hours
- [ ] Increase canary traffic step by step: 10% → 50% → 100%, only if metrics are healthy
- [ ] Collect first user feedback and support tickets daily
- [ ] Hot-fix process ready: small fix, fast review, same pipeline (never edit on the server)

Every week

- [ ] Review errors, slow queries and alerts; remove noisy alerts
- [ ] Apply dependency and security updates
- [ ] Track DORA metrics: deploy frequency, lead time, change failure rate, time to restore

Every month or quarter

- [ ] Backup restore drill and DR drill
- [ ] Access review: remove ex-employees and unused keys
- [ ] Cost review and right-sizing
- [ ] Re-run security scan; yearly VAPT
- [ ] Review compliance changes (DPDP full enforcement from May 2027, EU AI Act, CERT-In)

After every incident

- [ ] Blameless postmortem within 3 working days: what happened, timeline, root cause, fixes
- [ ] Action items have owners and due dates
- [ ] Add a test or alert so the same failure cannot repeat silently

Team training plan and sign-off

Train the team in four weekly 1-hour sessions, each ending with a hands-on exercise on a real project.

Week	Topic	Hands-on exercise
1	Before Development Checklist (separate doc) + safe AI coding	Each engineer writes a 1-page design doc and threat model for a small feature

Week	Topic	Hands-on exercise
2	Security (B3) + silly mistakes	Find and fix planted bugs: leaked secret, IDOR, missing timeout, SQL injection
3	Testing, CI/CD, release and rollback (B2, B7)	Do a canary deploy to staging, then roll it back
4	Observability, on-call and incidents (B5, B8, after launch)	Simulated outage drill + write a blameless postmortem

Launch sign-off

Area	Owner	Signed
Before Development Checklist signed off	Tech lead	
Code, testing and security (B1–B3)	Tech lead	
Infra, observability and release (B4, B5, B7)	DevOps owner	
Data and compliance (B6)	Product / legal	
Support and operations (B8)	Product owner	
Final go / no-go	Engineering manager	